



Universidad Nacional Experimental De Guayana

Vicerrectorado Académico.

Coordinación General De Pregrado.

Proyecto De Carrera: Ingeniería Informática

Lenguajes y Compiladores

Grupo: Control de Flujo

Los Lenguajes de Programación

Profesor:

Márquez Félix

Integrantes:

Canchis Victor: 27935276

Gonzalez Blas: 31074243

Muñoz Diosdado: 30932108

Rojas Celimar: 31981398

Ciudad Guayana, junio de 2026

Tabla de Contenido

Resumen Académico	7
Introducción	8
Actividad I. Matriz Descriptiva y Análisis de Paradigmática	9
Programación imperativa	10
<i>Beneficios de usar la programación imperativa</i>	<i>10</i>
<i>Limitaciones del uso de la programación imperativa</i>	<i>11</i>
<i>Buenas prácticas para el uso de la programación imperativa</i>	<i>12</i>
<i>¿Qué es la programación estructurada?</i>	<i>12</i>
<i>Beneficios de la programación estructurada</i>	<i>13</i>
<i>Desventajas de la programación estructurada</i>	<i>14</i>
<i>Gestión Explícita del Estado del Sistema</i>	<i>14</i>
<i>¿Cómo funciona la gestión explícita del estado?</i>	<i>14</i>
<i>Ventajas y Desventajas</i>	<i>15</i>
<i>Secuenciación de Instrucciones</i>	<i>15</i>
<i>Mutabilidad de memoria y efectos secundarios</i>	<i>16</i>
<i>Efectos Secundarios</i>	<i>17</i>
Paradigma Orientado a Objetos (POO)	18
<i>Modelos de encapsulamiento</i>	<i>19</i>
<i>Beneficios Principales</i>	<i>20</i>

<i>Polimorfismo</i>	20
<i>¿Cómo funciona el polimorfismo?</i>	21
<i>El polimorfismo tiene varios beneficios:</i>	21
<i>Herencia vs Composición</i>	21
<i>¿Cuál es la diferencia?</i>	22
<i>¿Cuál elegir?</i>	22
<i>Abstracción basada en datos y comportamiento combinados.</i>	23
<i>Ventajas de combinar ambos</i>	23
Matriz Descriptiva y Análisis del Paradigma Funcional	25
<i>El Paradigma Funcional: Fundamento Matemático e Infraestructura Lingüística</i>	25
<i>Funciones como Ciudadanos de Primer Orden (First-Class Functions) y Gestión de Memoria</i>	26
<i>Transparencia Referencial e Inmutabilidad: Implicaciones para la Optimización Estática</i>	27
<i>Evaluación Perezosa (Lazy Evaluation) vs. Evaluación Estricta (Eager Evaluation)</i>	28
<i>Convergencia Multiparadigma y su Justificación en la Ingeniería de Software</i>	30
Paradigma Lógico/Declarativo	31
<i>Programación basada en relaciones.</i>	31
<i>Unificación</i>	32
<i>Cláusulas de Horn</i>	33
<i>Resolución lógica</i>	34
<i>Abstracción del flujo de control</i>	34
<i>Marco Analítico</i>	35

Paradigma Concurrente / Modelo de Actores (Emergente)	36
<i>Modelos de paso de mensajes: comunicación desacoplada y ubicuidad del buzón.....</i>	<i>37</i>
<i>Aislamiento estricto de estado: encapsulación radical e inmutabilidad funcional</i>	<i>38</i>
<i>Mitigación de condiciones de carrera a nivel de diseño lingüístico</i>	<i>39</i>
<i>Convergencia multiparadigma: actores, objetos y funciones como respuesta al mercado</i>	<i>40</i>
Actividad II. Estudio Morfológico, Sintáctico y Benchmarking en Procesamiento Intensivo.....	44
Lógica Universal: Diagrama de Flujo Base	44
Implementación en Zig.....	46
<i>Análisis Morfológico (Léxico).....</i>	<i>46</i>
<i>Análisis Sintáctico.....</i>	<i>47</i>
Implementación en Python.....	49
<i>Análisis Morfológico (Léxico).....</i>	<i>49</i>
<i>Análisis Sintáctico (Gramática y Jerarquía).....</i>	<i>51</i>
Implementación en Rust.....	52
Implementación en Javascript	55
<i>Análisis Morfológico (Léxico).....</i>	<i>55</i>
<i>Análisis Sintáctico.....</i>	<i>56</i>
Benchmarking y Tabla de Tiempos	57
<i>Análisis comparativo de resultados</i>	<i>58</i>

Actividad III. Diseño de un Lenguaje de Dominio Específico (DSL) para Sistemas Lógico-Físicos Críticos.....	60
Los DSL como productos arquitectónicos	60
Especificación del Alfabeto y Reglas Léxicas	61
<i>Alfabeto y formato.....</i>	<i>61</i>
<i>Categorías de tokens.....</i>	<i>62</i>
<i>Tratamiento de elementos irrelevantes</i>	<i>63</i>
Palabras Clave Obligatorias y su Significado	63
<i>Comandos de inicialización y actuadores.....</i>	<i>64</i>
<i>Sensores</i>	<i>64</i>
<i>Estructuras de control.....</i>	<i>65</i>
<i>Constantes booleanas y operadores lógicos</i>	<i>65</i>
Gramática Sintáctica Abstracta	65
<i>Estructura General</i>	<i>66</i>
<i>Comandos</i>	<i>66</i>
<i>Asignación.....</i>	<i>66</i>
<i>Estructuras de control.....</i>	<i>67</i>
<i>Expresiones</i>	<i>67</i>
<i>Manejo de errores sintácticos.....</i>	<i>68</i>
Relación entre la gramática y la implementación	68
Programas de Ejemplo en Lenguaje L: Escenarios Operativos	69

<i>Escenario A – Prevención de Fuga Térmica y Gestión de Alivio de Carga</i>	<i>69</i>
<i>Escenario B – Balance de Carga y Optimización Energética Autónoma</i>	<i>70</i>
Conclusión	73
Referencias.....	74

Resumen Académico

Este documento presenta un análisis exhaustivo de los enfoques y metodologías que guían el desarrollo de software. Se exploran los principales paradigmas de programación, incluyendo el imperativo, estructurado, orientado a objetos, funcional, lógico y el modelo concurrente de actores, destacando la tendencia actual hacia la convergencia multiparadigma. Además, se realiza un estudio morfológico y sintáctico contrastando cuatro lenguajes (Zig, Python, Rust y JavaScript) mediante la implementación de un algoritmo universal para el cálculo de ecuaciones de segundo grado. Finalmente, el trabajo culmina con el diseño arquitectónico del "Lenguaje L", un Lenguaje de Dominio Específico (DSL) creado para operar un sistema crítico de microredes eléctricas (ECO-GRID), priorizando la legibilidad en español, la seguridad y la abstracción del hardware.

Introducción

La programación computacional se fundamenta en diversos estilos y métodos conocidos como paradigmas de programación, los cuales ofrecen distintas filosofías para resolver problemas. Aunque tradicionalmente se han clasificado de manera aislada, los lenguajes contemporáneos han evolucionado para integrar simultáneamente características de múltiples paradigmas en función de los desafíos que deban resolver.

El presente trabajo se divide en tres ejes analíticos y prácticos fundamentales:

- **Análisis Paradigmático:** Se describen los beneficios, limitaciones y funcionamientos internos de los paradigmas clásicos y emergentes, ilustrando cómo el mercado actual exige la hibridación de estas filosofías.
- **Benchmarking y Estudio Sintáctico:** Se examina cómo diferentes lenguajes (Zig, Python, Rust y JavaScript) procesan léxica y sintácticamente una misma lógica matemática estructurada a partir de un diagrama de flujo base universal.
- **Diseño de un DSL:** Se aborda la creación de un lenguaje a medida para un entorno industrializado complejo, demostrando cómo restringir la generalidad de un lenguaje de programación puede aumentar su expresividad y seguridad en dominios crítico.

Actividad I. Matriz Descriptiva y Análisis de Paradigmática

La programación engloba diferentes enfoques y técnicas guían el desarrollo de software. Estos enfoques, conocidos como paradigmas de programación, ofrecen distintas filosofías y metodologías para abordar problemas computacionales. Desde la clásica programación imperativa hasta los modernos paradigmas orientados a objetos y funcionales, cada uno tiene sus propias fortalezas, debilidades y aplicaciones específicas.

Un paradigma de programación es una manera o estilo de programación de software. Existen diferentes formas de diseñar un lenguaje de programación y varios modos de trabajar para obtener los resultados que necesitan los programadores. Se trata de un conjunto de métodos sistemáticos aplicables en todos los niveles del diseño de programas para resolver problemas computacionales.

Los lenguajes de programación adoptan uno o varios paradigmas en función del tipo de órdenes que permiten implementar como, por ejemplo, Python o JavaScript, que son multiparadigmas.

La clasificación tradicional de los paradigmas tiende a presentarlos como compartimentos estancos, sin embargo, los lenguajes contemporáneos de amplio propósito integran simultáneamente características funcionales, orientadas a objetos y concurrentes. A continuación se desarrollan cinco ejes temáticos, ilustrando cada paradigma con ejemplos de implementaciones reales y señalando las tensiones y sinergias que surgen de su integración.

Programación imperativa

La programación imperativa consiste en escribir programas como secuencias de comandos explícitos que se ejecutan en orden, de arriba abajo. El programador especifica los pasos exactos que debe seguir el ordenador, utilizando instrucciones como la asignación de variables, bucles y lógica condicional para controlar el flujo del programa y manipular los datos. Algunos de los lenguajes de programación imperativos mas comunes son:

- C/ C++
- Java
- C#
- JavaScript / TypeScript
- Python

Beneficios de usar la programación imperativa

La programación imperativa ofrece varias ventajas que la han convertido en un paradigma fundamental en la informática:

- Los lenguajes imperativos son relativamente fáciles de aprender porque el código se puede leer como instrucciones paso a paso.
- La sencillez del código imperativo resulta beneficiosa para el trabajo diario, ya que permite que diferentes empleados comprendan y mantengan el código, incluso si no lo escribieron inicialmente.

- La programación imperativa proporciona un control preciso sobre la ejecución del programa, lo que permite una gestión directa de la memoria y un flujo de control exacto para aplicaciones críticas en cuanto al rendimiento.
- Se utiliza con frecuencia para enseñar conceptos fundamentales de programación porque refleja la resolución de problemas paso a paso.

Limitaciones del uso de la programación imperativa

La programación imperativa, si bien ofrece varias ventajas, también conlleva ciertas limitaciones que los desarrolladores deben tener en cuenta:

- Para problemas complejos, el código imperativo puede volverse rápidamente extenso y difícil de entender debido a la cantidad de instrucciones necesarias para describir e implementar las soluciones.
- La ejecución no está claramente separada de la programación, lo que aumenta el riesgo de errores.
- Las extensiones son más difíciles de implementar en código puramente imperativo en comparación con el paradigma declarativo, porque extender la funcionalidad a menudo requiere modificar el código existente, mientras que la programación declarativa permite definir el comportamiento por separado.
- Los elementos del código suelen estar estrechamente interconectados. Esto significa que realizar cambios o correcciones de errores en una parte del programa puede tener repercusiones inesperadas en todo el sistema.

Buenas prácticas para el uso de la programación imperativa

Si bien la programación imperativa tiene sus inconvenientes, seguir las mejores prácticas puede ayudar a superar estos desafíos.

- Mantenga las funciones enfocadas y con un único propósito. Cada función debe tener una responsabilidad clara y realizar una sola tarea.
- Elige nombres para las variables y funciones que expresen claramente su propósito o contenido. Esto hace que el código sea más legible y se autodocumente.
- Evite las variables globales siempre que sea posible. Pueden dificultar la comprensión del código y provocar efectos secundarios no deseados.
- Divide los problemas en subproblemas más pequeños. Si un problema es demasiado complejo, divídelo en partes más pequeñas y manejables.
- Comprueba si hay entradas no válidas, gestiona los errores de forma adecuada y utiliza aserciones para documentar y verificar las suposiciones sobre el estado del programa.

¿Qué es la programación estructurada?

La programación estructurada es un paradigma informático que busca organizar el código de forma clara, ordenada y fácil de mantener. Su objetivo es mejorar la gestión de su complejidad, teniendo en cuenta las peculiaridades del pensamiento humano. Todo lo que es la programación estructurada está basado en el teorema esencial que determina que todo programa, sin importar el trabajo que realice, puede ser desarrollado con sólo las 3 estructuras básicas.

Estructura Secuencial: determina que las instrucciones de un programa se ejecuten una tras la otra. Es decir, en el mismo orden en el que aparecen en el programa.

Estructura Selectiva: propone 2 alternativas, basadas en el resultado de la evaluación de una condición. Lo que vendría a ser el equivalente a la instrucción IF que encontramos en todos los lenguajes de programación.

Estructura Repetitiva (Iterativa): vemos una ejecución repetida de alguna instrucción, mientras ocurre el cumplimiento de determinada condición.

Beneficios de la programación estructurada

1. Hace que los programas sean mucho más fáciles de entender, ya que pueden leerse de manera secuencial. Asimismo, te ahorras el rastreo de los saltos de línea en los bloques de código, para intentar comprender la lógica interna
2. Optimiza el esfuerzo durante las fases de prueba y depuración, haciendo más sencillo el seguimiento de los errores del programa y detectándolos rápidamente para su corrección.
3. Reduce los costos de mantenimiento, en esta fase, facilita la modificación o extensión de los programas de manera análoga a la depuración.
4. Plantea una estructura mucho más clara del programa, ya que las instrucciones suelen estar relacionadas o más ligadas entre sí.
5. Se puede ejecutar en cualquier equipo que cuente con un compilador o traductor acorde.

Desventajas de la programación estructurada

1. Al tener que traducir el lenguaje de alto nivel al de la máquina, sueles estar mucho tiempo en la computadora. Y esto, aunque no lo creas, puede resultar agotador.
2. Puede ocurrir que el código objeto que genera el compilador no sea del todo eficiente en caso de compararse con un programa similar en lenguaje ensamblador.
3. Tener el mismo código repetido en varias partes del programa puede hacer que el mismo sea vulnerable a problemas.

Gestión Explícita del Estado del Sistema

El paradigma imperativo es un enfoque de programación basado en instrucciones paso a paso que le indican a la computadora exactamente cómo resolver una tarea. Su núcleo es la gestión explícita del estado, lo que significa que el código manipula directamente la memoria y las variables del sistema a lo largo del tiempo

¿Cómo funciona la gestión explícita del estado?

En este paradigma, la memoria del programa (el estado) cambia continuamente a medida que se ejecutan las líneas de código.

- **Asignación de variables:** Guardas, modificas y actualizas datos constantemente en ubicaciones de memoria específicas.
- **Control de flujo:** Utilizas estructuras como bucles (for, while) y condicionales (if/else) para dictar el orden en que se altera el estado.

- **Efectos secundarios:** Las funciones pueden modificar variables globales o externas, lo que significa que el resultado de un bloque de código depende del momento exacto en que se ejecuta.

Ventajas y Desventajas

Ventajas	Desventajas
Control total: Tienes dominio absoluto sobre el uso de la memoria y el hardware.	Complejidad (Spaghetti Code): El estado cambia en múltiples lugares, lo que hace difícil rastrear errores.
Eficiencia: Permite optimizar al máximo el rendimiento y la velocidad de ejecución.	Concurrencia difícil: Si varios procesos intentan modificar el estado a la vez, pueden surgir graves problemas.
Lectura secuencial: Es muy intuitivo para modelar procesos paso a paso en la vida real.	Menos predecible: El mismo código puede dar resultados distintos dependiendo de los cambios previos en el sistema.

Secuenciación de Instrucciones

1. **Ejecución lineal:** Las instrucciones se escriben y ejecutan de arriba hacia abajo.
2. **Modificación de estado:** Se utilizan variables para almacenar, actualizar y manipular datos a lo largo de todo el ciclo de vida del programa.
3. **Estructuras de control:** Rompen la linealidad pura mediante condicionales (decisiones) y bucles (repeticiones).

4. **Enfoque procedural:** Frecuentemente se divide el código en subrutinas o funciones para organizar las secuencias

Ventajas	Desventajas
Intuición: Sigue el pensamiento lógico humano de "hacer las cosas paso a paso".	Complejidad: El código puede volverse difícil de mantener a medida que el proyecto crece (código espagueti).
Control: Ofrece control directo y preciso sobre los recursos del sistema y la memoria.	Mutabilidad: Al cambiar el estado de las variables constantemente, es más propenso a errores imprevistos.

Mutabilidad de memoria y efectos secundarios

Los programas son secuencias de comandos que modifican el estado de la memoria a través de variables mutables. Esto facilita el control sobre el hardware, pero introduce problemas de mantenimiento y depuración debido a los efectos secundarios. En este paradigma, una variable es una dirección de memoria física o virtual cuyo valor puede cambiar (mutar) a lo largo del tiempo de ejecución.

- **Asignación destructiva:** El valor previo de una variable se pierde y se sobrescribe por uno nuevo.
- **Estado cambiante:** El programa avanza mediante la alteración continua de estos bloques de memoria

Efectos Secundarios

Un efecto secundario ocurre cuando una función o bloque de código modifica un estado o variable externa a su propio entorno local.

- **Comportamiento no local:** Una función en la línea 50 puede cambiar una variable declarada en la línea 10.
- **Pérdida de transparencia referencial:** Al depender del estado mutable global, llamar a una misma función con los mismos parámetros puede devolver resultados distintos dependiendo de en qué momento del programa se ejecute

La combinación de memoria mutable y efectos secundarios presenta ventajas y desventajas críticas que impactan la calidad del software.

Pros	Contras
Mayor control de hardware: Permite interactuar directamente con la memoria, ideal para sistemas embebidos o videojuegos.	Código difícil de seguir: Para entender un programa, es necesario leer el código paso a paso y recordar mentalmente qué variables cambiaron y dónde.
Optimización de recursos: Evita crear copias innecesarias de datos reutilizando espacios de memoria ya reservados.	Errores imprevistos (Bugs): Modificar un estado compartido puede alterar accidentalmente el comportamiento de otras partes del programa.

Ejecución secuencial: La lógica paso a paso es muy intuitiva para problemas algorítmicos secuenciales y directos.	Complejidad concurrente: En programas paralelos, si varios hilos acceden y modifican la misma memoria, pueden causar condiciones de carrera y resultados corruptos.
---	---

Hoy en día, usamos lenguajes orientados a objetos, funcionales o declarativos (como SQL o React) para manejar la complejidad a gran escala y ocultar el estado. Pero en los motores de las bases de datos y en los compiladores, el código sigue siendo estrictamente imperativo y estructurado. Todo paradigma de alto nivel, al final del día, debe traducirse a instrucciones imperativas y secuenciales para que el procesador pueda ejecutarlo.

Paradigma Orientado a Objetos (POO)

La POO es un paradigma que ha ganado una gran popularidad en los últimos años debido a su capacidad para crear aplicaciones más robustas, flexibles y fáciles de mantener. Esta metodología de desarrollo se basa en la idea de que los programas se pueden organizar como una colección de objetos interconectados, cada uno con su propio conjunto de datos y funcionalidades.

Con el paradigma de Programación Orientado a Objetos lo que buscamos es dejar de centrarnos en la lógica pura de los programas, para empezar a pensar en objetos, lo que constituye la base de este paradigma. Esto nos ayuda muchísimo en sistemas grandes, ya que,

en vez de pensar en funciones, pensamos en las relaciones o interacciones de los diferentes componentes del sistema.

Un programador diseña un programa de software organizando piezas de información y comportamientos relacionados en una plantilla llamada clase. Luego, se crean objetos individuales a partir de la plantilla de clase. Todo el programa de software se ejecuta haciendo que varios objetos interactúen entre sí para crear un programa más grande.

La POO se inspira en la forma en que percibimos y entendemos el mundo que nos rodea. En este enfoque, cada una de estas entidades se convierte en un objeto, con propiedades (datos) y comportamientos (funcionalidades). La clave de la POO radica en la interacción entre estos objetos. Pueden comunicarse entre sí enviándose mensajes y colaborando para lograr un objetivo común.

Modelos de encapsulamiento

La encapsulación contiene toda la información importante de un objeto dentro del mismo y solo expone la información seleccionada al mundo exterior. Esta propiedad permite asegurar que la información de un objeto esté oculta para el mundo exterior, agrupando en una Clase las características o atributos que cuentan con un acceso privado, y los comportamientos o métodos que presentan un acceso público.

La encapsulación de cada objeto es responsable de su propia información y de su propio estado. La única forma en la que este se puede modificar es mediante los propios métodos del objeto. Por lo tanto, los atributos internos de un objeto deberían ser inaccesibles desde fuera, pudiéndose modificar sólo llamando a las funciones correspondientes. Con esto conseguimos mantener el estado a salvo de usos indebidos o que puedan resultar inesperados.

El encapsulamiento se implementa principalmente utilizando modificadores de acceso. Estos definen el modelo de visibilidad de los atributos y métodos:

- **Público (Public):** Los atributos y métodos son accesibles desde cualquier lugar, tanto dentro como fuera de la clase. Ofrece mayor flexibilidad, pero menor seguridad, ya que expone los datos directamente.
- **Privado (Private):** Es el pilar del encapsulamiento. Los miembros solo son accesibles dentro de la propia clase que los define. Protege el estado interno del objeto impidiendo modificaciones directas desde el exterior.
- **Protegido (Protected):** Los miembros son accesibles dentro de la propia clase y también por sus clases hijas (herencia), pero no desde fuera de la jerarquía de clases

Beneficios Principales

- **Protección de datos:** Evita que otros objetos corrompan los datos internos al no poder alterarlos directamente.
- **Modularidad y Mantenimiento:** La implementación interna de la clase puede cambiar (por ejemplo, cambiar el tipo de dato o la lógica de un cálculo) sin que afecte al código externo que la utiliza.
- **Control:** Permite restringir los atributos a solo lectura si no se proporciona un método setter.

Polimorfismo

El polimorfismo consiste en diseñar objetos para compartir comportamientos, lo que nos permite procesar objetos de diferentes maneras. Es la capacidad de presentar la misma

interfaz para diferentes formas subyacentes o tipos de datos. Al utilizar la herencia, los objetos pueden anular los comportamientos principales compartidos, con comportamientos secundarios específicos. El polimorfismo permite que el mismo método ejecute diferentes comportamientos de dos formas: anulación de método y sobrecarga de método.

¿Cómo funciona el polimorfismo?

El polimorfismo se basa en establecer relaciones entre clases a través de la herencia. Cuando una superclass define un método, sus subclasses pueden sobrescribirlo para proporcionar su propia implementación. En tiempo de ejecución, se llama al método correcto según el tipo real del objeto. Este proceso, conocido como enlace dinámico, permite que tu código sea más adaptable y extensible.

El polimorfismo tiene varios beneficios:

Reutilización de código y modularidad: Las clases pueden compartir características comunes a través de la herencia.

- **Flexibilidad:** Puedes añadir nuevas subclasses sin modificar el código existente.
- **Algoritmos genéricos:** Es posible escribir algoritmos que trabajen con diferentes tipos de objetos, haciendo tu código más simple y eficiente.

Herencia vs Composición

Herencia es cuando un objeto o clase se basa en otro objeto o clase, usando la misma implementación o comportamiento. Esto es un mecanismo para la reutilización de código

para permitirnos extensiones independientes del software original mediante clases públicas e interfaces. La herencia tiene cabida donde existe una relación “es un/una”.

Composición: la composición es un principio de diseño en el que un objeto se compone de otros objetos. En lugar de heredar propiedades y comportamientos de una clase padre (como en la herencia), se crea un comportamiento complejo combinando varios objetos simples. Este patrón de diseño es increíblemente útil porque fomenta la flexibilidad, la modularidad y la reutilización.

¿Cuál es la diferencia?

Si bien la herencia es ideal para modelar relaciones jerárquicas, la composición ofrece una forma más modular y flexible de combinar diferentes funcionalidades. La herencia crea un acoplamiento estrecho entre las clases padre e hija, lo que puede dificultar la modificación o extensión del sistema a medida que crece. En cambio, la composición permite construir relaciones más flexibles y sin restricciones.

¿Cuál elegir?

La herencia está fuertemente acoplada, mientras que la composición está débilmente acoplada.

En la herencia no existe control de acceso, mientras que en la composición se puede restringir. Exponemos todos los métodos de la superclase a las demás clases que tienen acceso a la subclase. Por lo tanto, si se introduce un nuevo método o existen vulnerabilidades de seguridad en la superclase, la subclase se vuelve vulnerable. Dado que en la composición elegimos qué métodos usar, es más segura que la herencia.

La composición proporciona flexibilidad en la invocación de métodos, lo cual es útil en escenarios con múltiples subclases.

Abstracción basada en datos y comportamiento combinados

La abstracción es el pilar que permite crear modelos conceptuales. Al unir datos (información o atributos) y comportamiento (acciones o métodos), aísla la lógica esencial de los detalles técnicos complejos. Esto permite interactuar con los objetos a través de interfaces claras y simplificadas.

- **Abstracción de Datos:** Define qué información es relevante para el sistema (estado).
- **Abstracción de Comportamiento:** Define qué acciones puede realizar el objeto o qué se le puede pedir que haga (funcionalidad).

Para combinar estos dos aspectos, los lenguajes orientados a objetos (como Java, Python, C++ o C#) utilizan principalmente dos herramientas:

- **Clases Abstractas:** Son plantillas incompletas que definen atributos (datos) y métodos estándar, pero dejan ciertos métodos sin implementar (abstractos) para que las subclases definan el comportamiento específico.
- **Interfaces:** Ocultan totalmente los datos (no tienen variables de estado directas) y definen un conjunto de firmas de métodos. La clase que implementa la interfaz está obligada a proveer el comportamiento.

Ventajas de combinar ambos

- **Modularidad:** Los cambios internos en la implementación de un método no afectan al resto del programa siempre que la interfaz (el "qué hace") se mantenga.

- **Reducción de complejidad:** Permite a los programadores trabajar a un nivel conceptual más alto sin preocuparse por la complejidad de bajo nivel.
- **Reutilización de código:** Una vez abstraído el comportamiento y los datos, el modelo se puede reutilizar y extender para crear objetos más específicos.

La Programación Orientada a Objetos es actualmente el paradigma que más se utiliza para diseñar aplicaciones y programas informáticos. Son muchas sus ventajas, principalmente cuando necesitas resolver desafíos de programación complejos. Permite una mejor estructura de datos y reutilización del código, lo que facilita el ahorro de tiempo a largo plazo. Para ello se requiere pensar bien en la estructura del programa, planificar al comienzo de la codificación, así como analizar los requisitos en clases simples y reutilizables que se pueden usar para diseñar instancias de objetos.

El desarrollo de software moderno ya no se trata de debatir cuál es el paradigma "correcto". Se trata de tener la madurez técnica para combinar las mejores herramientas de cada enfoque, construyendo sistemas que no solo sean rápidos y escalables, sino que puedan ser leídos y mantenidos por equipos humanos a largo plazo.

El mercado define la herramienta: La necesidad de procesar grandes volúmenes de datos, acelerar los tiempos de entrega y manejar la concurrencia en la nube obligó a los lenguajes a abandonar el purismo en favor de la resolución real de problemas.

- **La sinergia estructural:** La ingeniería moderna ha encontrado su punto de equilibrio al utilizar la POO para diseñar el "esqueleto" del sistema (arquitectura, encapsulamiento e inyección de dependencias) y la PF para el "sistema nervioso" (transformación inmutable de datos y lógica de negocio).

- **Mayor poder exige mayor disciplina:** Al integrar múltiples paradigmas, los lenguajes ofrecen una flexibilidad sin precedentes, pero transfieren la responsabilidad de mantener el código limpio, coherente y libre de caos (código Frankenstein) directamente a los arquitectos y equipos de desarrollo.

Matriz Descriptiva y Análisis del Paradigma Funcional

El Paradigma Funcional: Fundamento Matemático e Infraestructura Lingüística

El paradigma funcional no es una alternativa sintáctica superficial; es una traducción directa del Cálculo Lambda (λ -cálculo) desarrollado por Alonzo Church en la década de 1930 para el estudio formal de la computabilidad. Desde la perspectiva de la teoría de compiladores, un lenguaje funcional puro opera como azúcar sintáctico sobre el λ -cálculo, reduciendo la semántica de la ejecución a reglas formales de α -conversión (renombramiento de variables ligadas para evitar colisiones de alcance) y β -reducción (aplicación de una función a sus argumentos mediante sustitución de variables).

En este paradigma computacional, se cumplen tres propiedades lingüísticas fundamentales:

- **Omnipresencia funcional:** Toda expresión válida constituye una función, eliminando la distinción imperativa entre enunciados (statements) que modifican el estado y expresiones que devuelven valores.
- **Sustitución pura:** La computación avanza exclusivamente mediante la aplicación secuencial de funciones y la sustitución matemática de sus términos.

- **Ausencia de estado mutable:** El diseño no contempla la asignación destructiva de memoria en tiempo de ejecución.

Funciones como Ciudadanos de Primer Orden (First-Class Functions) y Gestión de Memoria

Que las funciones sean ciudadanos de primer orden implica que comparten los mismos privilegios operativos que cualquier tipo de dato primitivo (enteros, flotantes, etc.). En consecuencia, el compilador debe admitir que una función se asigne a variables locales, se pase como argumento o se retorne desde otra función. Este último caso produce las Funciones de Orden Superior (Higher-Order Functions).

Para soportar esta flexibilidad, la arquitectura del compilador o intérprete altera el comportamiento convencional de la memoria de la siguiente manera:

Mecanismo Técnico	Comportamiento en la Arquitectura de Memoria	Impacto en Compiladores / Intérpretes
Clausuras (Closures)	Cuando una función de orden superior retorna una función interna, el entorno léxico capturado (variables del ámbito externo) se empaqueta junto al código de la función.	El compilador genera una estructura de datos explícita como par ordenado (código, entorno).

Promoción al Heap	A diferencia del paradigma imperativo clásico, las variables locales capturadas por una closure no pueden destruirse al finalizar el marco de activación de la pila (stack frame).	El ciclo de vida de esos datos trasciende el bloque original; el compilador los promueve al heap para garantizar su persistencia.
Currificación (Currying)	Transformación de una función con múltiples argumentos en una cadena de funciones unarias.	Permite la aplicación parcial, reteniendo los argumentos iniciales en el heap mediante referencias activas hasta completar la evaluación.

Transparencia Referencial e Inmutabilidad: Implicaciones para la Optimización Estática

Una expresión exhibe transparencia referencial si puede sustituirse por su valor evaluado en cualquier punto del flujo de ejecución sin alterar el comportamiento semántico del programa. Formalmente, si una función f con entrada x produce un resultado v :

$f(x) \equiv v \Rightarrow$ Sustituir $f(x)$ por v preserva la invariabilidad del sistema

Esta propiedad exige la ausencia de efectos colaterales (side effects): el compilador asume que no existe mutabilidad de memoria global, modificación de punteros ni interacción directa no encapsulada con operaciones de E/S.

Esta restricción habilita las siguientes optimizaciones estáticas, inaccesibles en entornos puramente mutables:

- **Memorización nativa:** Si $f(x)$ no tiene efectos secundarios, el compilador puede almacenar en caché el resultado tras la primera llamada. Las invocaciones posteriores con el mismo argumento se resuelven en tiempo constante $O(1)$, sin reejecutar el cuerpo de la función.
- **Plegado de constantes (Constant Folding):** Las llamadas a funciones transparentes con argumentos constantes se evalúan durante la fase de optimización, antes de la generación de código nativo, y se sustituyen directamente por sus valores computados.
- **Reordenamiento e instrucciones paralelas:** Al eliminar dependencias destructivas de memoria, el compilador puede reordenar libremente las instrucciones de evaluación o asignar subexpresiones a hilos concurrentes sin riesgo de condiciones de carrera (race conditions) ni necesidad de bloqueos mutuos (locks).
- **Eliminación de código muerto (Dead Code Elimination):** Si el análisis estático determina que el resultado de una expresión pura no es consumido por ninguna rutina posterior, esa expresión se descarta bajo la certeza de que no afecta la integridad del programa.

Evaluación Perezosa (Lazy Evaluation) vs. Evaluación Estricta (Eager Evaluation)

La evaluación perezosa (o call-by-need) difiere el cómputo de una expresión hasta el momento exacto en que otra instrucción requiere su valor. El mecanismo central son los Thunks: objetos en memoria que encapsulan la expresión aún no evaluada junto con las referencias de su entorno léxico.

La comparación entre ambas estrategias revela diferencias concretas en rendimiento y gestión de memoria:

Dimensión Técnica	Evaluación Estricta (Eager / Call-by-Value)	Evaluación Perezosa (Lazy / Call-by-Need)
Instante de cómputo	Inmediato. Los argumentos se evalúan por completo antes de ingresar al cuerpo de la función.	Diferido. El argumento entra a la función como thunk y se computa solo si es forzado.
Gestión de memoria	Demanda inicial mayor y predecible. Todos los valores intermedios se almacenan en el momento de su generación.	Demanda inicial menor, con riesgo de fragmentación por acumulación de thunks pendientes en el heap.
Compartición de cómputo	No aplica. Al copiar una variable se transfieren datos ya procesados.	Una vez forzado el thunk, el valor resultante reemplaza al objeto en el heap (memorización implícita).
Estructuras de datos	Acotadas y finitas. Intentar instanciar una estructura de tamaño infinito produce desbordamiento de pila o bloqueo.	Soporta estructuras infinitas (streams, listas infinitas) evaluadas bajo demanda sin desbordamiento.
Depuración (Debugging)	Lineal y predecible. Las trazas de pila (stack traces) corresponden directamente a la secuencia lógica de ejecución.	Compleja. Los errores se manifiestan en el punto de consumo del valor, desacoplados del punto de definición.

Convergencia Multiparadigma y su Justificación en la Ingeniería de Software

La adopción de estrategias funcionales en lenguajes de propósito general —como Java 8+, Python, JavaScript, C# y Kotlin— responde a límites concretos del hardware actual y a la Ley de Amdahl. Con el fin del escalado de frecuencia en procesadores mononúcleo, el rendimiento del software depende de su capacidad para aprovechar arquitecturas multinúcleo (multicore).

Los lenguajes generales integran herramientas del paradigma funcional por tres razones estructurales:

1. **Abstracción sobre flujos de concurrencia:** El uso de funciones de orden superior en interfaces fluidas —como la Stream API en Java o LINQ en C#— permite declarar operaciones de transformación (mapas, filtros, reducciones) sin gestionar manualmente hilos de ejecución. El compilador asume la responsabilidad de fragmentar e inyectar las tareas en paralelo de forma segura.
2. **Reducción de errores por estado compartido:** Introducir estructuras inmutables nativas —como `val` en Kotlin u objetos congelados en JavaScript— disminuye los fallos por estado compartido en aplicaciones distribuidas, y permite transferir datos entre procesos sin la sobrecarga que imponen los mecanismos tradicionales de sincronización y semáforos.
3. **Optimización guiada por semántica funcional:** Al incorporar funciones flecha y tipado estático funcional, los motores modernos —como V8 en JavaScript— identifican con mayor facilidad las secciones de código susceptibles de

optimizaciones Just-In-Time (JIT), incluida la inyección directa en línea (inlining) de funciones pequeñas y sin efectos secundarios.

Paradigma Lógico/Declarativo

El paradigma lógico o declarativo representa una forma de programación radicalmente distinta a los enfoques imperativos tradicionales. Mientras que en los lenguajes imperativos el programador especifica paso a paso cómo debe resolverse un problema, en el paradigma lógico se describe únicamente qué condiciones deben cumplirse para obtener una solución.

Este paradigma se fundamenta en la lógica matemática, particularmente en la lógica de predicados de primer orden, donde los programas se expresan mediante hechos, reglas y consultas. El programador define relaciones entre los datos y el sistema de ejecución se encarga de deducir automáticamente las respuestas mediante mecanismos formales de inferencia.

Por esta razón, el paradigma lógico es considerado uno de los máximos exponentes de la programación declarativa, ya que el control de ejecución se encuentra ampliamente abstraído y es administrado por el motor lógico del lenguaje.

Programación basada en relaciones

La unidad fundamental de la programación lógica son las relaciones. En lugar de modelar procedimientos o secuencias de instrucciones, se describen relaciones existentes entre distintos elementos del dominio del problema. Por ejemplo:

padre(carlos, juan).

padre(carlos, maria).

Estas expresiones indican relaciones entre individuos, estableciendo que Carlos es padre de Juan y de María.

hermano(X,Y) :-

padre(Z,X),

padre(Z,Y),

$X \neq Y$.

La regla expresa que dos personas son hermanas si comparten el mismo padre y son distintas entre sí. Lo importante es que el programador no establece un algoritmo específico para encontrar hermanos, sino que simplemente declara las relaciones existentes y el sistema deduce automáticamente las respuestas. Esta característica permite construir programas altamente expresivos y cercanos al razonamiento humano.

Unificación

La unificación es el mecanismo que permite al sistema encontrar coincidencias entre consultas, hechos y reglas almacenados en la base de conocimiento. Su objetivo es determinar si dos expresiones pueden hacerse equivalentes mediante la sustitución adecuada de variables. Este proceso constituye uno de los pilares fundamentales de la programación lógica, ya que permite automatizar gran parte del razonamiento que en otros paradigmas debe ser programado manualmente. Por ejemplo, la consulta:

padre(carlos, X), puede unificarse con:

padre(carlos, juan), obteniendo como resultado:

$$X = \text{juan}$$

Durante la ejecución, el motor lógico compara continuamente consultas con hechos y reglas almacenados en la base de conocimiento; si tiene éxito, la búsqueda continúa, y si falla, el sistema explora otras alternativas. La importancia de este mecanismo radica en que sustituye el trabajo algorítmico tradicional, permitiendo que la resolución de problemas se base en coincidencias lógicas en lugar de instrucciones explícitas.

Cláusulas de Horn

Las cláusulas de Horn constituyen la estructura formal utilizada para representar reglas en la programación lógica. Estas cláusulas establecen una conclusión que será válida siempre que se cumplan determinadas condiciones previas. Su importancia radica en que proporcionan una base matemática sólida para la construcción de sistemas capaces de realizar inferencias automáticas, traducándose en la práctica sintáctica de lenguajes como Prolog en reglas de la forma:

$$H \leftarrow B_1 \wedge B_2 \wedge \cdots \wedge B_n$$

Por ejemplo:

```
abuelo(X,Y) :-  
    padre(X,Z),  
    padre(Z,Y).
```

Esta regla indica que una persona será abuelo de otra cuando exista un individuo intermedio que sea hijo del primero y padre del segundo. A partir de este tipo de estructuras, el motor lógico puede aplicar procesos de resolución para determinar si una consulta es verdadera o falsa.

Resolución lógica

La resolución es el procedimiento mediante el cual el sistema intenta demostrar si una consulta es verdadera a partir de los hechos y reglas disponibles. Cuando un usuario realiza una consulta, el motor lógico inicia un proceso de búsqueda sistemática para encontrar evidencias que satisfagan dicha petición, permitiendo que el lenguaje realice razonamientos automáticos complejos sin que el programador deba implementar algoritmos de búsqueda, recorridos o estructuras de control explícitas. Por ejemplo:

?- abuelo(carlos, maria).

El sistema buscará reglas relacionadas con "abuelo", intentará unificar variables, verificará hechos existentes y continuará encadenando inferencias hasta demostrar la consulta o determinar que no existe solución. Este mecanismo se conoce como resolución por refutación y constituye uno de los pilares fundamentales de la inteligencia artificial simbólica.

Abstracción del flujo de control

La abstracción del flujo de control es la delegación total de la ejecución al motor de inferencia, diferenciándose del paradigma imperativo en que el programador no define el

"cómo" (ciclos, orden o estados), sino el "qué" (hechos y reglas). Este enfoque se fundamenta en el postulado de Robert Kowalski

Algoritmo = Lógica + Control

Donde el desarrollador solo asume la componente lógica, permitiendo que el motor gestione el control mediante un flujo declarativo sin variables mutables ni efectos secundarios, y donde la iteración se logra exclusivamente a través de la recursión abstracta en la pila de resolución.

Por ejemplo, al realizar una consulta en Prolog:

```
?- hermano(juan, X) .
```

El programador no indica cómo recorrer la base de datos ni qué estrategia de búsqueda utilizar, sino que es el intérprete quien decide automáticamente cómo explorar las alternativas, aplicar unificaciones y realizar retrocesos (*backtracking*) hasta encontrar todas las soluciones posibles.

Marco Analítico

El paradigma lógico aporta una forma diferente de abordar la programación, ya que se enfoca en describir relaciones y reglas para que el sistema encuentre las soluciones de manera automática. Aunque los lenguajes puramente lógicos no son los más utilizados en el desarrollo de software actual, sus principios siguen teniendo una gran influencia en áreas como la inteligencia artificial, los sistemas expertos y los motores de búsqueda basados en conocimiento.

Actualmente, la industria se inclina hacia lenguajes multiparadigma capaces de combinar diferentes enfoques según las necesidades del problema. Por esta razón, lenguajes

como Python, JavaScript y Rust incorporan características provenientes de distintos paradigmas, incluyendo algunas ideas inspiradas en la programación lógica, funcional y orientada a objetos. Esta convergencia permite desarrollar aplicaciones más flexibles, mantenibles y adaptadas a los retos modernos del software.

Cabe destacar que el paradigma lógico no debe verse como una tecnología aislada o limitada a ciertos lenguajes, sino como una fuente de conceptos que han contribuido a la evolución de la programación moderna. Su importancia radica en haber introducido mecanismos de razonamiento e inferencia que continúan presentes, de una u otra forma, en muchas de las herramientas y lenguajes utilizados en la actualidad.

Paradigma Concurrente / Modelo de Actores (Emergente)

El Modelo de Actores, propuesto inicialmente por Carl Hewitt en 1973, ha resurgido como un paradigma emergente fundamental para abordar la complejidad de los sistemas modernos, los cuales son inherentemente distribuidos, altamente escalables y de alta concurrencia. En este paradigma, el «actor» es la primitiva universal de la computación concurrente: una entidad computacional autónoma que encapsula estado y comportamiento, y que interactúa con otros actores exclusivamente a través del paso de mensajes asíncronos. Frente a la complejidad de la gestión manual de hilos, bloqueos y memoria compartida, el modelo de actores propone una arquitectura donde la concurrencia se aborda desde el diseño mismo del lenguaje o del framework, eliminando por construcción las condiciones de carrera y proporcionando un modelo mental más cercano a los sistemas distribuidos reales.

A continuación, se desarrolla el análisis conceptual basado en los tres ejes temáticos requeridos y se examina el fenómeno contemporáneo de la convergencia multiparadigma.

Modelos de paso de mensajes: comunicación desacoplada y ubicuidad del buzón

En el paradigma de actores, el paso de mensajes constituye el único mecanismo de interacción entre entidades computacionales. A diferencia de los modelos de concurrencia clásicos basados en hilos que comparten memoria, los actores no invocan métodos de otros actores de forma directa ni comparten referencias a su estado interno. En su lugar, cada actor posee un buzón (mailbox) que actúa como frontera asíncrona: los mensajes se encolan y son procesados secuencialmente, uno a uno, por el actor receptor. El emisor simplemente deposita el mensaje en el buzón del destinatario y continúa su ejecución sin bloquearse.

Este esquema introduce un desacoplamiento espacio-temporal radical. El emisor no necesita conocer la ubicación física del receptor, ni su estado, ni siquiera si está activo en ese instante. La comunicación es inherentemente asíncrona y basada en eventos, lo que maximiza el rendimiento del procesador y permite la ejecución de tareas de forma verdaderamente paralela y reactiva. La semántica del paso de mensajes se define lingüísticamente con operaciones de envío no bloqueante (send, !) y recepción que puede implementarse como pattern matching sobre la forma del mensaje, como ocurre en Erlang y Elixir.

El impacto arquitectónico es profundo. Al eliminar las llamadas a procedimientos síncronos entre componentes concurrentes, se suprime la dependencia inmediata de resultados y se habilita la composición de flujos reactivos. Además, el buzón puede ser persistente o transaccional (como en sistemas de mensajería empresarial), difuminando la frontera entre concurrencia local y mensajería distribuida. Esta propiedad facilita la transición

de un prototipo mononúcleo a un clúster geodistribuido sin reescribir la lógica de negocio, un atributo esencial en la era de la computación en la nube.

Aislamiento estricto de estado: encapsulación radical e inmutabilidad funcional

El segundo eje definitorio es el aislamiento total del estado mutable: cada actor mantiene su estado privado y ningún otro actor puede leerlo o modificarlo directamente. Esta encapsulación es más estricta que la encapsulación clásica de la orientación a objetos, porque ni siquiera existen referencias compartidas a la memoria de un actor; la única vía para consultar o transformar ese estado es mediante el envío de mensajes, que serán procesados secuencialmente por el propio actor, preservando la integridad.

Cuando un actor procesa un mensaje de su buzón (lo cual hace de forma secuencial, un mensaje a la vez), puede realizar tres acciones fundamentales:

- Tomar decisiones locales y modificar su propio estado interno.
- Enviar mensajes a otros actores.
- Crear nuevos actores.

Al garantizar que el estado de un actor solo puede mutar como reacción al procesamiento de un mensaje, se elimina la imprevisibilidad del acceso concurrente. Esta propiedad converge naturalmente con el paradigma funcional: la inmutabilidad de los mensajes y, frecuentemente, del estado interno del actor (que se transforma produciendo una nueva versión del estado, como en las funciones reductoras) elimina los efectos laterales no controlados y simplifica el razonamiento sobre el comportamiento. En Erlang, las variables son inmutables y el estado de un proceso (actor) se mantiene mediante recursión de cola con parámetros actualizados; en Akka, los actores pueden mutar su estado interno de forma

controlada, pero las mejores prácticas recomiendan tratarlo como valor inmutable y utilizar estructuras persistentes.

El aislamiento estricto es también la base de la tolerancia a fallos y la escalabilidad. Dado que un actor solo puede fallar aisladamente, los sistemas de actores implementan jerarquías de supervisión: un actor padre monitoriza a sus hijos y decide si reiniciarlos, detenerlos o escalar el fallo, siguiendo una estrategia declarativa. Esta propiedad es explotada por lenguajes y frameworks como Erlang/OTP, donde el modelo de «let it crash» asume que los fallos son inevitables y la recuperación se gestiona a nivel de actor, sin necesidad de complejas transacciones distribuidas. El aislamiento también permite que actores se distribuyan transparentemente entre nodos de un clúster, ya que su estado privado es autocontenido y la localización se resuelve mediante el sistema de mensajes.

Mitigación de condiciones de carrera a nivel de diseño lingüístico

El modelo de actores traslada la prevención de errores de concurrencia desde el esfuerzo manual del programador hacia la arquitectura misma del lenguaje o framework. En la programación multihilo tradicional, el acceso a la memoria compartida requiere primitivas de sincronización complejas (como mutexes, semáforos o locks), lo que a menudo deriva en interbloqueos (deadlocks) o condiciones de carrera (race conditions). Por el contrario, al prohibir la memoria compartida mutable y procesar los mensajes de un actor de forma estrictamente secuencial, el paradigma elimina por diseño la posibilidad de que dos hilos intenten modificar la misma variable simultáneamente. El lenguaje impone una topología donde la condición de carrera simplemente no tiene espacio para existir.

No obstante, a nivel de lógica de negocio pueden surgir carreras de orden entre mensajes (por ejemplo, si dos mensajes llegan en un orden imprevisto que altera el resultado esperado). La mitigación se aborda mediante el diseño de protocolos basados en estado y máquinas de estados finitas que absorben la no determinación temporal. Herramientas lingüísticas como el pattern matching sobre la forma del mensaje y el estado actual del actor (por ejemplo, cláusulas receive que solo aceptan ciertos mensajes en un estado dado) permiten diferir el procesamiento hasta que el sistema esté en el estado apropiado. Algunos lenguajes ofrecen tipado de sesión para protocolos de comunicación: Akka Typed, por ejemplo, impone protocolos a nivel de tipos, verificando qué mensajes se pueden enviar a un actor según su estado comportamental.

Adicionalmente, lenguajes como Pony llevan la mitigación un paso más allá mediante un sistema de tipos con capacidades de referencia (reference capabilities) que garantiza, en tiempo de compilación, la ausencia de data races y de alias mutables concurrentes, incluso entre actores que comparten memoria en el mismo proceso. Esto demuestra que es posible combinar un rendimiento cercano al metal con seguridad de concurrencia sin depender exclusivamente del aislamiento por buzón, aunque siempre manteniendo la filosofía de no compartir estado mutable.

Convergencia multiparadigma: actores, objetos y funciones como respuesta al mercado

En el panorama contemporáneo, el desarrollo de software raramente se ciñe a un lenguaje de nicho puramente orientado a actores. Para responder a las demandas del mercado que exigen desarrollo rápido, ecosistemas maduros y escalabilidad cloud-native, lenguajes de amplio propósito general han adoptado el Modelo de Actores, convergiendo con paradigmas

Orientados a Objetos (POO) y Funcionales (PF). Lejos de presentarse como un paradigma aislado, los actores se hibridan con POO y PF para ofrecer soluciones pragmáticas que atienden las necesidades de sistemas reactivos de alta concurrencia, microservicios y computación en la nube.

Convergencia con la orientación a objetos. La analogía entre un actor y un objeto que encapsula estado y solo responde a mensajes es natural. Sin embargo, mientras que en la POO tradicional la invocación de métodos es síncrona y los objetos comparten referencias, los actores imponen asincronía y aislamiento absoluto. Frameworks como Akka (en la JVM) o Microsoft Orleans (.NET) permiten definir actores como clases, aprovechando herencia, polimorfismo y patrones de diseño conocidos, pero añadiendo el buzón y la ejecución secuencial. De este modo, equipos con experiencia en POO pueden migrar progresivamente hacia arquitecturas concurrentes sin abandonar su ecosistema. Scala, con Akka, es el ejemplo por excelencia de esta convergencia: fusiona POO y PF de forma nativa en la JVM, pero a través de Akka implementa el Modelo de Actores de forma robusta; un actor se define como una clase (POO), procesa mensajes utilizando pattern matching (PF) y escala masivamente sin bloquear hilos del sistema operativo (Concurrente).

Convergencia con la programación funcional. El estado aislado e inmutable de los actores y los mensajes como valores sin efectos laterales casan perfectamente con los principios funcionales. Erlang es el ejemplo canónico: un lenguaje funcional puro (sin datos mutables) donde los procesos (actores) se crean, se comunican y mueren en un entorno de alta disponibilidad. Elixir retoma esta herencia y la moderniza con metaprogramación y pipelines funcionales. En Scala, Akka se beneficia del uso de case classes inmutables y del

pattern matching funcional para definir el comportamiento de los actores, logrando una sinergia expresiva. Esta confluencia permite que los desarrolladores apliquen razonamiento ecuacional en la lógica de cada actor, mientras la concurrencia se abstrae en las interacciones.

Adopción en lenguajes comerciales y de sistemas. Recientemente, Apple introdujo la palabra clave actor como un tipo base en Swift (junto a class y struct), demostrando cómo un lenguaje comercial e imperativo/orientado a objetos adopta protección de estado concurrente a nivel de compilador. Un actor en Swift protege automáticamente su estado mutable, obligando al programador a usar la sintaxis await para interactuar con él, fusionando la POO tradicional con la seguridad concurrente emergente. En el ámbito de sistemas, aunque Go utiliza canales y el modelo CSP (Communicating Sequential Processes) en lugar del modelo de actores puro, y Rust emplea sistemas de propiedad (ownership) para garantizar la seguridad de la memoria en concurrencia, ambos reflejan la misma tendencia multiparadigma: dotar a lenguajes pragmáticos de características de aislamiento de estado a nivel de diseño lingüístico para evitar condiciones de carrera sin sacrificar el rendimiento.

Convergencia y demandas del mercado. La necesidad de gestionar millones de conexiones simultáneas, ofrecer baja latencia y alta resiliencia (como en sistemas de mensajería instantánea, IoT o plataformas de streaming) ha impulsado la adopción del modelo de actores más allá de los nichos tradicionales. El éxito de WhatsApp, construido sobre Erlang/FreeBSD, evidenció su viabilidad. Hoy, el ecosistema cloud-native incorpora actores virtuales (Orleans) que abstraen la localización y el ciclo de vida, y la JVM dispone de Akka, que combina actores con streams reactivos y programación funcional. Incluso lenguajes de propósito general como Rust o Go han popularizado el paso de mensajes como

patrón idiomático (canales y gorutinas en Go), acercándose a la filosofía de «no compartir memoria, comunicar», lo que demuestra la influencia del paradigma en el diseño lingüístico actual.

En definitiva, el paradigma concurrente de actores, al elevar la mitigación de condiciones de carrera, el aislamiento de estado y el paso de mensajes a elementos de diseño del lenguaje, proporciona un marco conceptual robusto. Su integración con la orientación a objetos y la programación funcional ejemplifica la convergencia multiparadigma que define el desarrollo de software contemporáneo, permitiendo que los lenguajes de amplio propósito respondan con soluciones escalables, seguras y mantenibles ante las crecientes exigencias de los sistemas distribuidos modernos.

Actividad II. Estudio Morfológico, Sintáctico y Benchmarking en Procesamiento Intensivo

Para contrastar filosóficamente lenguajes de propósito general, se utilizaron cuatro entornos con aproximaciones tecnológicas profundamente diferenciadas. Zig representa el enfoque moderno de sistemas, diseñado para sustituir a C, con control explícito de memoria, ausencia de variables ocultas y metaprogramación en tiempo de compilación (comptime). Python, lenguaje interpretado de alto nivel de sintaxis clara, domina la ciencia de datos e IA aunque sufre penalizaciones por el Global Interpreter Lock (GIL). Rust, lenguaje compilado de sistemas, impone un modelo de propiedad (ownership) que garantiza seguridad de memoria sin recolector de basura, compitiendo en velocidad con C++. JavaScript, originalmente diseñado para la web, se ejecuta sobre motores como V8 y ha extendido su modelo asíncrono no bloqueante al lado del servidor mediante Node.js. El estudio se organiza en tres niveles: análisis morfológico (léxico), análisis sintáctico (estructuras de control) y benchmarking cuantitativo.

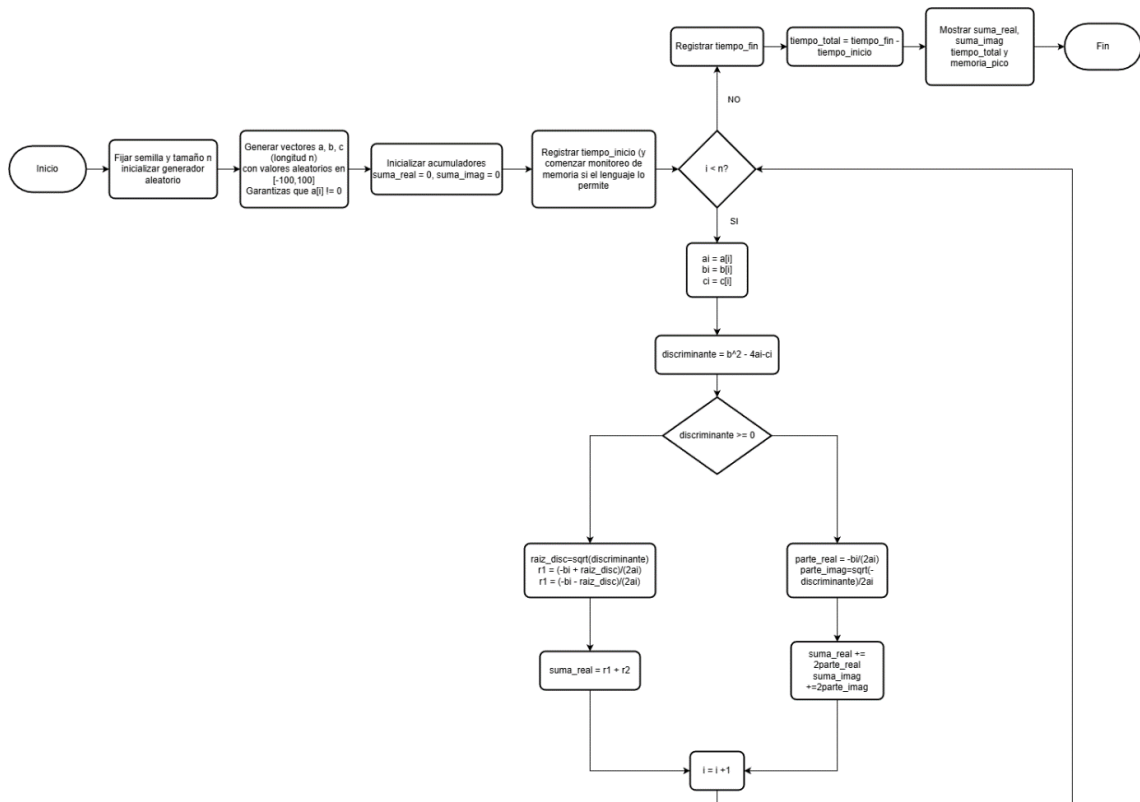
Para la correcta comparación de los lenguajes se eligió uno de tres programas para realizar en todos los lenguajes, el cual era el siguiente:

“Cálculo de ecuación de segundo grado para vectores a,b,c con cantidad de elemento n para n=200”

Lógica Universal: Diagrama de Flujo Base

Para garantizar la consistencia algorítmica y permitir una comparativa justa entre las distintas tecnologías, se diseñó un diagrama de flujo universal previo a la codificación. Esta

representación gráfica actuó como el plano arquitectónico base para los cuatro lenguajes de programación. El diagrama estandariza la inicialización determinista de los vectores a, b, c de longitud $n=200$ mediante una semilla estática, el cálculo del discriminante (para discriminar entre raíces reales y complejas) y la acumulación de resultados. Al transcribir fielmente esta lógica a cada lenguaje, aseguramos que las variaciones observadas en los tiempos de ejecución y la memoria pico (como los medidos en milisegundos y MB) dependan exclusivamente de la arquitectura del compilador o intérprete subyacente, y no de discrepancias en la aproximación del problema matemático.



Implementación en Zig

Análisis Morfológico (Léxico)

Zig es un lenguaje de programación de sistemas de paradigma imperativo/estructurado, que incorpora soporte para metaprogramación en tiempo de compilación y algunas características funcionales (funciones de primera clase, inmutabilidad por defecto). Su estructura léxica se caracteriza por:

- Palabras reservadas: `const`, `var`, `fn`, `pub`, `try`, `defer`, `while`, `if`, `else`, `return`, `struct`, `enum`, `union`, `switch`, `break`, `continue`, `true`, `false`, `null`, `undefined`, `error`, `usingnamespace`, `align`, `allowzero`, `asm`, `async`, `await`, `comptime`, `errdefer`, `extern`, `inline`, `noalias`, `noinline`, `nosuspend`, `opaque`, `packed`, `resume`, `suspend`, `threadlocal`, `volatile`. En el código presentado se utilizan estrictamente: `const`, `var`, `fn`, `pub`, `try`, `defer`, `while`, `if`, `else`.
- Identificadores: Comienzan con una letra o guion bajo, seguidos de letras, dígitos o guion bajo. Sensibles a mayúsculas/minúsculas. Ejemplos en el código: `queryPerformanceFrequency`, `debug_allocator`, `suma_real`, `init`, `r_state`.
- Literales:
 - Enteros: `200`, `1505171219`, `1664525`, `1013904223`, `0xFFFFFFFF`.
 - Flotantes: `0.0`, `1.0`, `-100.0`, `200.0`, `4.0`, `2.0`, `1_000_000_000.0`, `1_000_000.0`.
 - Booleanos: `true`, `false` (evaluados internamente mediante el método `.toBool()`).
- Delimitadores y comentarios:
 - Llaves `{ }` para bloques de alcance (scopes).

- Paréntesis () para expresiones y llamadas a funciones.
- Punto y coma ; como terminador obligatorio de sentencias.
- Comentarios de línea: //.
- Tratamiento de elementos irrelevantes: Zig ignora espacios en blanco y saltos de línea para la estructuración del árbol de sintaxis abstracta; la indentación es un elemento estético de legibilidad ya que los bloques se delimitan exclusivamente por llaves.

Análisis Sintáctico

La sintaxis de Zig está diseñada para eliminar la ambigüedad en el parsing. A continuación, se describen las estructuras de control presentes en el código del benchmark, documentadas mediante la notación formal Backus-Naur Form (BNF):

1. Estructura de un programa Zig

```
programa ::= ( decl_const | funcion )*
```

2. Declaración de constantes

```
decl_const ::= "const" identificador ":" tipo "=" expresion ";"
```

Ejemplo del código: *const n: usize = 200;*

3. Definición de funciones

```
funcion ::= "pub"? "fn" identificador "(" parametros ")" "!" tipo  
bloque
```

Ejemplo del código: *fn queryPerformanceFrequency() !u64 { ... }*

4. Bucle while (Estructura iterativa)

```
while_bucle ::= "while" "(" expresion ")" ":" expresion bloque
```

El código utiliza este patrón con actualización explícita en la cabecera para mitigar bucles infinitos por descuido:

```
while (idx < n) : (idx += 1) { ... }
while (i < n) : (i += 1) { ... }
```

5. Condicional if-else

```
if_else ::= "if" "(" expresion ")" bloque ( "else" bloque )?
```

Ejemplo del código:

```
if (disc >= 0.0) {
    // raíces reales
} else {
    // raíces complejas
}
```

6. Manejo de errores con try

```
try_expresion ::= "try" expresion
```

Utilizado para la propagación explícita de errores hacia el llamador superior, como en: `try queryPerformanceFrequency();`.

7. Asignación y liberación de memoria

```
asignacion_memoria ::= "try" allocator "." "alloc" "(" tipo ","
expresion ")"
liberacion          ::= "defer" allocator "." "free" "(" identificador
")"
```

El par operacional garantiza que la liberación del Heap ocurra inmediatamente al cerrarse el ciclo de vida del bloque actual, emulando un comportamiento seguro contra fugas de memoria (memory leaks).

8. Jerarquía sintáctica en el cálculo de raíces

Las expresiones aritméticas son parseadas respetando la precedencia de operadores (multiplicación y división antes que suma/resta). Los paréntesis actúan como nodos prioritarios en el Árbol de Sintaxis Abstracta (AST):

$$r1 = (-bi + \text{sqrt_disc}) / (2.0 * ai)$$

El analizador sintáctico ramifica esta expresión colocando al operador divisor (/) como raíz, bifurcando en un subárbol izquierdo para el numerador $(-bi + \text{sqrt_disc})$ y uno derecho para el denominador $(2.0 * ai)$.

Implementación en Python

A continuación, se desglosa el comportamiento del intérprete de Python (CPython) al procesar el archivo `vectores.py`, analizando cómo la máquina abstrae el texto fuente desde sus caracteres más básicos hasta las estructuras de control complejas.

Análisis Morfológico (Léxico)

Es fundamental destacar que la tokenización en compiladores y lenguajes formales es estrictamente determinista y se basa en expresiones regulares y autómatas finitos, asignando categorías gramaticales exactas (ej. `IDENTIFICADOR`, `FLOTANTE`). Esto difiere radicalmente de la tokenización probabilística (como *Byte-Pair Encoding*) utilizada en los modelos de lenguaje (LLMs) para el procesamiento de lenguaje natural.

En el código Python analizado, el analizador léxico (`lex`) categoriza los elementos bajo las siguientes reglas:

- **Palabras Reservadas (Keywords):** Son tokens exclusivos que dictan la estructura del programa. En este script, el lexer identifica `import`, `for`, `in`, `while`, `if`, y `else`.
- **Identificadores:** Secuencias utilizadas para nombrar variables o módulos. Python exige que comiencen con una letra (a-z, A-Z) o guion bajo `_`, seguidos de letras, números o guiones bajos.
 - *Ejemplos en el código:* `a_val`, `suma_real`, `discriminante`, `tracemalloc`, `math`.
- **Literales Soportados (Datos en crudo):**
 - *Numéricos Enteros:* Secuencias de dígitos (`200`, `1505171219`, `4`).
 - *Numéricos Flotantes:* Números que incluyen un punto decimal o notación científica (`0.0`, `-100.0`, `-1e-12`, `1e-12`).
 - *Cadenas de Texto (F-Strings):* El lexer de Python trata a las cadenas precedidas por `f` (ej. `f"Tiempo de cálculo: {tiempo_ms:.6f} ms"`) como tokens especiales que requieren evaluación de variables incrustadas durante la ejecución.
- **Elementos Irrelevantes y Delimitadores:**
 - *Comentarios:* Cualquier texto precedido por `#` (como `#Configuracion` o `#Vectores`) es completamente descartado por el lexer y no genera tokens.
 - *Indentación Significativa:* A diferencia de lenguajes como C o Java que usan llaves `{ }` para delimitar bloques, el lexer de Python rastrea los niveles de sangría. Emite tokens virtuales ocultos llamados `INDENT` (cuando se entra a un bloque `for` o `if`) y `DEDENT` (cuando se regresa al nivel anterior). Los saltos de

línea `\n` actúan como delimitadores explícitos de final de instrucción (`NEWLINE`), eliminando la necesidad de puntos y comas `;`.

Análisis Sintáctico (Gramática y Jerarquía)

Una vez que el código se convierte en una lista de tokens, el analizador sintáctico (Parser) de Python construye el Árbol de Sintaxis Abstracta (AST). A continuación, se documenta mediante **Notación de Forma de Backus-Naur Extendida (EBNF)** la jerarquía gramatical de las tres estructuras de control principales utilizadas en `vectores.py`.

A. Estructura Iterativa Definida (`for` loop)

Utilizada para la iteración sobre el rango `$n=200$` en la generación de vectores y el cálculo de raíces.

```
<for_stmt> ::= "for" <target_list> "in" <expression_list> ":" <suite>
<target_list> ::= <identifier> (ej. "i")
<expression_list> ::= <function_call> (ej. "range(n)")
<suite> ::= <NEWLINE> <INDENT> <statement>+ <DEDENT>
```

Jerarquía lógica: El bloque `for` exige una expresión iterable y delega la ejecución al bloque `<suite>`, que contiene las declaraciones anidadas tabuladas.

B. Estructura Iterativa Indefinida (`while` loop)

Empleada para la corrección térmica del vector `$a$` (evitando el valor `$0.0$` absoluto mediante la condición estricta `-1e-12 < a_val < 1e-12`).

```
<while_stmt> ::= "while" <expression> ":" <suite>
<expression> ::= <comparison> (ej. "-1e-12 < a_val < 1e-12")
<suite> ::= <NEWLINE> <INDENT> <statement>+ <DEDENT>
```

C. Estructura Condicional Bifurcada (`if / else`)

Implementada para evaluar el discriminante matemático y derivar el flujo de cálculo hacia raíces reales o imaginarias conjugadas.

```
<if_stmt> ::= "if" <expression> ":" <suite>
           ["else" ":" <suite>]
<expression> ::= <identifier> <operator> <literal>
(ej. "discriminante >= 0")
```

Jerarquía lógica: El nodo `<if_stmt>` crea un árbol binario en la sintaxis. Si la `<expression>` evalúa a `True`, el AST dirige la ejecución a la primera `<suite>` (cálculo con `math.sqrt(discriminante)`). Si evalúa a `False`, el parser exige la presencia del token `else` para dirigir la ejecución a la segunda `<suite>` correspondiente al cálculo complejo.

Descripción del algoritmo implementado

El algoritmo desarrollado en Rust resuelve ecuaciones de segundo grado utilizando tres vectores de coeficientes (a, b y c) con un tamaño de **n = 200**. Para cada conjunto de coeficientes se calcula el discriminante y se determina si las raíces son reales o imaginarias. Posteriormente, las soluciones obtenidas son acumuladas en variables de suma, permitiendo generar una carga de procesamiento adecuada para realizar pruebas de rendimiento. Finalmente, el programa mide el tiempo total de ejecución con el fin de utilizar los resultados en el proceso de benchmarking entre los distintos lenguajes analizados.

Implementación en Rust

Análisis Morfológico (Léxico)

Como lenguaje compilado de sistemas orientado al rendimiento y seguridad de memoria, el compilador de Rust transforma el código fuente en una secuencia de tokens. Los componentes léxicos principales identificados en el programa son:

- **Palabras reservadas:** Como `use`, `struct`, `impl`, `fn`, `let`, `mut`, `for`, `while` e `if`, las cuales poseen un significado específico dentro del lenguaje y no pueden utilizarse como identificadores.
- **Identificadores:** Siguen la convención `snake_case` característica de Rust, observándose nombres como `suma_real`, `suma_imag`, `tiempo_inicio`, `discriminante` y `gen_range`.
- **Literales:** Se emplean enteros como `200` y `1505171219`, empleados para definir el tamaño de los vectores y la semilla del generador de números aleatorios. Asimismo, se emplean literales de punto flotante como `100.0`, `4.0` y `1e-12`, necesarios para realizar operaciones matemáticas sobre números reales.
- **Operadores:** se utilizan operadores aritméticos (`+`, `-`, `*`, `/`), operadores de asignación (`=` y `+=`), operadores de comparación (`>=`, `<`) y operadores de acceso mediante el punto (`.`), usados para invocar métodos asociados a estructuras y tipos de datos.
- **Delimitadores y Espacios:** A diferencia de lenguajes como Python, Rust ignora la indentación significativa y los espacios en blanco. Utiliza explícitamente llaves `{ }` para definir los bloques de código y el alcance de funciones, bucles y condicionales.

Análisis Sintáctico

La estructura sintáctica del programa se rige por la gramática formal de Rust, organizando sus componentes jerárquicamente a partir de la función principal `main()`:

- **Estructuras e Implementación:** Se declara una estructura denominada `SimpleRng`, encargada de implementar un generador de números aleatorios mediante el método LCG (Linear Congruential Generator). Posteriormente, mediante un bloque `impl`, se definen los métodos asociados a dicha estructura, permitiendo crear instancias y generar valores aleatorios dentro de rangos específicos.
- **Configuración y Vectores:** Dentro de la función principal se establecen las variables de configuración, incluyendo el valor de $n = 200$ y la semilla utilizada para garantizar la reproducibilidad de los resultados. Luego se crean tres vectores (`a`, `b` y `c`) que almacenan los coeficientes de las ecuaciones cuadráticas.
- **Control de Flujo de Datos (Bucle `for` y `while`):** La asignación de coeficientes utiliza un ciclo `for` jerárquico. Adicionalmente, se anida una sentencia condicional `while` encargada de evaluar que los valores del vector `a` sean distintos de cero, evitando divisiones inválidas.
- **Procesamiento:** El cálculo central se ejecuta mediante un ciclo `for`. Para cada conjunto de coeficientes se calcula el discriminante de la ecuación cuadrática. Posteriormente se emplea una estructura condicional `if-else` que determina si las raíces son reales o imaginarias. Cuando el discriminante es mayor o igual a cero, se calculan dos raíces reales utilizando la fórmula general. En caso contrario, se calculan las componentes real e imaginaria correspondientes.
- **Salida:** El programa registra el tiempo de ejecución utilizando la biblioteca estándar de Rust y muestra por pantalla las sumatorias obtenidas junto con el tiempo total requerido para completar el procesamiento.

Implementacion en Javascript

Análisis Morfológico (Léxico)

Como lenguaje interpretado y de tipado dinámico (ejecutado bajo el entorno de Node.js), el intérprete transforma el código fuente en una secuencia de tokens. Los componentes léxicos principales identificados en el programa son:

- **Palabras reservadas:** Como ``const``, ``let``, ``function``, ``return``, ``for``, ``while`` e ``if``, las cuales poseen un significado específico dentro del lenguaje para el control de flujo y declaración en memoria, y no pueden utilizarse como identificadores.
- **Identificadores:** Siguen en su mayoría la convención camelCase característica de JavaScript, observándose nombres como ``crearRNG``, ``generarVectores``, ``ejecutarAlgoritmo`` y ``memoriaPico``, aunque también se integran identificadores en snake_case como ``suma_real`` y ``suma_imag``.
- **Literales:** Se emplean enteros como ``200``, ``12345``, ``10`` y ``100``, empleados para definir el tamaño de los vectores, la semilla del generador de números aleatorios y las iteraciones del benchmark. Asimismo, se emplean literales para operaciones matemáticas (como ``2``, ``4``) y notación científica (``1e6``), necesarios para conversiones numéricas. También destacan los literales de plantilla (``...``) para formatear cadenas de texto.
- **Operadores:** Se utilizan operadores aritméticos (``+``, ``-``, ``*``, ``/``, ``%``), operadores de asignación (``=``, ``+=``), operadores de comparación (``>=``, ``<``, ``===``) y operadores de

acceso mediante el punto (`.`), usados para invocar métodos asociados a objetos nativos del lenguaje como `Math` y `process`.

- **Delimitadores y Espacios:** A diferencia de lenguajes como Python, JavaScript ignora la indentación significativa y los espacios en blanco. Utiliza explícitamente llaves `{ }` para definir los bloques de código y el alcance de funciones, bucles y condicionales, así como el punto y coma (`;`) para denotar la finalización de una instrucción.

Análisis Sintáctico

La estructura sintáctica del programa se rige por la gramática formal de JavaScript (ECMAScript), organizando sus componentes jerárquicamente a partir del llamado a la función principal `ejecutarBenchmark()`:

- **Funciones y Clausuras:** En lugar de estructuras formales, se declara una función de orden superior denominada `crearRNG`, encargada de implementar un generador de números aleatorios mediante el método LCG (Linear Congruential Generator). Mediante el uso de una clausura (closure), se retorna una función interna que mantiene el estado de la semilla a través de las diferentes llamadas.
- **Configuración y Vectores:** En el ámbito superior se establecen las constantes de configuración, incluyendo el valor de `N = 200` y la semilla utilizada para garantizar la reproducibilidad de los resultados. Luego se crean tres arreglos (`a`, `b` y `c`) que almacenan los coeficientes de las ecuaciones cuadráticas creados mediante la función `generarVectores`.

- **Control de Flujo de Datos (Bucle for y while):** La asignación de coeficientes utiliza un ciclo ``for`` iterativo. Adicionalmente, se anida una sentencia condicional ``while`` encargada de evaluar que los valores del vector ``a`` sean estrictamente distintos de cero (``val === 0``), evitando divisiones inválidas.
- **Procesamiento:** El cálculo central se ejecuta en la función ``ejecutarAlgoritmo`` mediante un ciclo ``for``. Para cada conjunto de coeficientes se extraen los valores iterados y se calcula el discriminante de la ecuación cuadrática utilizando el objeto nativo ``Math``. Posteriormente se emplea una estructura condicional ``if-else`` que determina si las raíces son reales o imaginarias. Cuando el discriminante es mayor o igual a cero, se calculan dos raíces reales utilizando la fórmula general. En caso contrario, se calculan las componentes real e imaginaria correspondientes sumándolas a los acumuladores.
- **Salida:** El programa registra el tiempo de ejecución y el consumo de memoria utilizando los módulos nativos del entorno de Node.js (``process.hrtime.bigint`` y ``process.memoryUsage``) y muestra por pantalla las sumatorias obtenidas junto con el tiempo promedio y la memoria pico requerida para completar el procesamiento.

Benchmarking y Tabla de Tiempos

El benchmarking proporciona una herramienta comparativa relevante para el ingeniero de software de alto nivel. Para este estudio se seleccionó la demostración de la conjetura de Collatz para todos los números menores que n ($n=200$) porque genera una carga de procesamiento intensiva predecible, con bucles anidados y operaciones aritméticas que

<i>Lenguaje de Programación</i>	<i>Paradigma Dominante</i>	<i>Mecanismo de Ejecución y Compilación</i>	<i>Tiempo de Ejecución Promedio (ms)</i>	<i>Consumo de Memoria Pico (MB)</i>
Zig	<i>Imperativo / Estructurado</i>	<i>Compilación Nativa (LLVM / Backend propio) sin GC</i>	<i>0.0053 ms</i>	<i>0.004578 MB</i>
Python	<i>Multiparadigma (OO, Imperativo)</i>	<i>Compilación a Bytecode e Interpretado por VM (CPython)</i>	<i>5.968300 ms</i>	<i>0.019203 MB</i>
Rust	<i>Multiparadigma (Funcional, Imperativo)</i>	<i>Compilación Nativa (LLVM)</i>	<i>0.0576 ms</i>	<i>0.0043 MB</i>
JavaScript	<i>Multiparadigma (Prototípico, Funcional)</i>	<i>JIT (Just-In Time) / V8 Engin</i>	<i>0.2829 ms</i>	<i>2.0266 MB</i>

ejercitan tanto la CPU como la gestión de pila. Las condiciones controladas fueron: hardware con procesador Intel Core i7-18665U, 12 GB RAM DDR4, Sistema Operativo Windows 11 Pro. La tabla presenta los tiempos medios de ejecución:

Análisis comparativo de resultados

Los resultados obtenidos evidencian diferencias significativas entre los lenguajes evaluados en términos de tiempo de ejecución y consumo de memoria, las cuales están directamente relacionadas con su paradigma y mecanismo de compilación. En cuanto al rendimiento, **Zig** presentó el menor tiempo de ejecución promedio (0.0053 ms), seguido por

Rust (0.0576 ms), demostrando la ventaja de los lenguajes compilados de forma nativa sobre aquellos que dependen de máquinas virtuales o motores de ejecución intermedios. Por su parte, **JavaScript** obtuvo un desempeño intermedio (0.2829 ms) gracias a la optimización proporcionada por el motor V8 y la compilación JIT, mientras que **Python** registró el mayor tiempo de ejecución (5.968300 ms), debido a su naturaleza interpretada y al uso de bytecode ejecutado por la máquina virtual de CPython.

Respecto al consumo de memoria, **Rust** y **Zig** mostraron los valores más bajos, con 0.0043 MB y 0.004578 MB respectivamente, reflejando una gestión eficiente de recursos al no depender de mecanismos de ejecución adicionales. En contraste, **Python** incrementó ligeramente su consumo de memoria (0.019203 MB), mientras que **JavaScript** presentó una diferencia considerablemente mayor, alcanzando 2.0266 MB, resultado asociado a la carga del entorno de ejecución y a las optimizaciones internas del motor V8.

De forma general, la comparación permite concluir que los lenguajes compilados nativamente, como Zig y Rust, ofrecen un mejor equilibrio entre velocidad y eficiencia en el uso de memoria. JavaScript logra un rendimiento aceptable gracias a la compilación JIT, aunque con un costo elevado en recursos de memoria, mientras que Python prioriza la facilidad de desarrollo y la productividad del programador sobre la optimización del rendimiento. Estos resultados confirman cómo las características internas de cada lenguaje influyen directamente en su comportamiento durante la ejecución de programas.

Actividad III. Diseño de un Lenguaje de Dominio Específico (DSL) para Sistemas Lógico-Físicos Críticos

La consolidación de los conceptos gramaticales se materializa al asumir el rol de diseñador de lenguajes para un dominio acotado. En este caso, se plantea un problema de Interfaz Hombre-Máquina (HMI) en un entorno industrializado complejo: un Sistema de Gestión de Microredes Eléctricas Inteligentes y Almacenamiento de Energía (ECO-GRID). La microred integra paneles solares, turbinas eólicas, bancos de baterías de litio, sensores de flujo eléctrico y caudal de carga, sensores térmicos de celda y conmutadores electrónicos (relés de alta potencia). El lenguaje propuesto, denominado Lenguaje L, está orientado a operadores humanos y debe permitir monitorizar, controlar y optimizar la red sin exponer la complejidad del hardware subyacente, pero manteniendo suficiente expresividad para responder a fallos críticos.

Los DSL como productos arquitectónicos

Un Lenguaje de Dominio Específico (DSL) es un lenguaje de programación diseñado para un ámbito concreto de aplicación, a diferencia de los lenguajes de propósito general (GPL) como Python o C. Mientras un GPL debe servir para cualquier tarea, un DSL sacrifica generalidad para ganar expresividad, seguridad y cercanía al vocabulario del experto del dominio.

El DSL se concibe así como un producto arquitectónico: no es simplemente una herramienta, sino una interfaz que traduce las intenciones del operador humano en comandos ejecutables por la máquina, ocultando la complejidad subyacente.

En el caso del sistema ECO-GRID, el dominio es una microred eléctrica con generación renovable, baterías, sensores y actuadores de alta potencia. Los operadores necesitan leer variables físicas (temperatura, carga, flujo de energía) y ejecutar acciones de control (conmutar líneas, activar refrigeración, emitir alertas). El Lenguaje L se diseña para que estas tareas se expresen con una sintaxis en español, cercana al lenguaje natural estructurado, y con un conjunto mínimo de construcciones.

Especificación del Alfabeto y Reglas Léxicas

El análisis léxico convierte una secuencia de caracteres en una secuencia de tokens (unidades con significado gramatical). Las reglas léxicas de L se han definido para ser simples, no ambiguas y orientadas al dominio.

Alfabeto y formato

- **Codificación:** UTF-8 (compatible ASCII). Se distinguen mayúsculas y minúsculas.
- **Separadores:** Espacios y tabulaciones son ignorados (excepto como separadores de tokens). El salto de línea (`\n`) actúa como terminador de sentencia principal; el punto y coma no es necesario pero puede usarse.
- **Comentarios:** desde `//` hasta el final de la línea. El lexer los descarta sin generar tokens.

Categorías de tokens

Categoría	Ejemplos	Descripción
Palabras reservadas	init_grid, si_verdadero, mientras, conmutar_linea, leer_temperatura, verdadero, falso	Comandos del sistema, estructuras de control, constantes booleanas y operadores lógicos en español.
Identificadores	bateria_1, sector_A, temp	Nombres de variables, baterías o sectores. Comienzan con letra o <code>_</code> , seguidos de letras, dígitos o <code>_</code> .
Literales numéricos	200, 55.0, 1e-3	Enteros (secuencia de dígitos) y flotantes (con punto decimal obligatorio).
Operadores	+, -, *, /, ==, !=, <, >, <=, >=	Aritméticos y relacionales.
Operadores lógicos	y, o, no	En español.
Asignación	=	Asocia un valor a una variable.

Delimitadores	(,), ,	Paréntesis para funciones y agrupación, coma para separar argumentos.
Fin de sentencia	NUEVA_LINEA	El salto de línea delimita las sentencias.

El lexer (implementado con PLY) está definido en el código mediante expresiones regulares (líneas 87-117). La función `t_ID` distingue palabras reservadas de identificadores usando un diccionario (`palabras_reservadas`).

Tratamiento de elementos irrelevantes

- **Espacios y tabulaciones:** ignorados (`t_ignore = '\t'`).
- **Comentarios:** ignorados (`t_comentario`).
- **Salto de línea:** generan el token `NUEVA_LINEA`, que el parser usa como separador de sentencias.
- **Errores léxicos:** caracteres no reconocidos disparan `t_error`, que informa la línea y omite el carácter.

Palabras Clave Obligatorias y su Significado

El lenguaje L incorpora palabras reservadas explícitas que reflejan directamente las operaciones sobre el sistema físico y las estructuras de control necesarias para la automatización.

Comandos de inicialización y actuadores

Palabra reservada	Función
init_grid	Inicializa el bus de datos de la microred.
conmutar_linea(sector, estado)	Abre (falso) o cierra (verdadero) un relé de alta potencia.
activar_refrigeracion(bateria, estado)	Enciende o apaga el sistema de enfriamiento auxiliar de una batería.
emitir_alerta	Activa una alarma visual/sonora en la HMI.
esperar(segundos)	Pausa la ejecución durante un tiempo determinado (útil para ciclos de monitoreo).

Sensores

Palabra reservada	Retorno	Función
leer_temperatura(bateria)	Flotante (°C)	Lee el sensor térmico de una celda.
estado_carga(bateria)	Flotante (%)	Lee el nivel de carga de una batería.

leer_generacion(fuente)	Flotante (kW)	Lee la potencia generada por paneles solares o turbinas.
leer_demanda(sector)	Flotante (kW)	Lee la potencia consumida por un sector.

Estructuras de control

Palabra reservada	Descripción
si_verdadero ... entonces ... [sino ...] fin_si	Condicional.
mientras ... ejecutar ... fin_mientras	Bucle condicional.
repetir ... veces ... fin_repetir	Bucle con número fijo de iteraciones.

Constantes booleanas y operadores lógicos

- verdadero, falso
- y, o, no

Estas palabras reservadas están definidas en el diccionario `palabras_reservadas` y son reconocidas por el lexer como tokens específicos (SI, ENTONCES, CONMUTAR, etc.).

Gramática Sintáctica Abstracta

La sintaxis del lenguaje L se define mediante una gramática libre de contexto no ambigua, implementada con un parser LALR(1) generado por PLY. La gramática establece el orden lógico que debe seguir una sentencia para que el intérprete la ejecute sin ambigüedad.

Estructura General

```

programa          ::= lista_sentencias

lista_sentencias  ::= sentencia | lista_sentencias sentencia

sentencia         ::= comando NEWLINE
                   | asignacion NEWLINE
                   | declaracion_si
                   | declaracion_mientras
                   | declaracion_repetir
                   | NEWLINE

```

Un programa en L es una secuencia de sentencias. Cada sentencia es un comando, una asignación o una estructura de control, y finaliza con un salto de línea.

Comandos

```

comando ::= 'init_grid'
         | 'conmutar_linea' '(' ID ',' expresion ')'
         | 'activar_refrigeracion' '(' ID ',' expresion ')'
         | 'emitir_alerta'
         | 'esperar' '(' expresion ')'

```

Los comandos de actuación toman argumentos: un identificador de recurso y (si aplica) una expresión booleana o numérica.

Asignación

```

asignacion ::= ID '=' expression

```

Permite almacenar el resultado de una expresión (p. ej., lectura de un sensor) en una variable.

Estructuras de control

```

    declaracion_si    ::= 'si_verdadero' expresion 'entonces' NEWLINE
lista_sentencias 'fin_si'
                    | 'si_verdadero' expresion 'entonces' NEWLINE
lista_sentencias 'sino' NEWLINE lista_sentencias 'fin_si'
    declaracion_mientras ::= 'mientras' expresion 'ejecutar' NEWLINE
lista_sentencias 'fin_mientras'
    declaracion_repetir ::= 'repetir' expresion 'veces' NEWLINE
lista_sentencias 'fin_repetir'

```

Las estructuras de control delimitan sus bloques con palabras clave de apertura y cierre, sin dependencia de la indentación (aunque se recomienda usarla para legibilidad).

Expresiones

La gramática de expresiones sigue una jerarquía de precedencia estándar:

```

expresion            ::= expresion_logica
expresion_logica     ::= expresion_relacional (('y' | 'o')
expresion_relacional)*
expresion_relacional ::= expresion_aritmetica
(('==' | '!=' | '<' | '>' | '<=' | '>=') expresion_aritmetica)?
expresion_aritmetica ::= termino (('+' | '-') termino)*
termino              ::= factor (('*' | '/') factor)*
factor               ::= ENTERO | FLOTANTE | VERDADERO | FALSO
                    | ID
                    | 'leer_temperatura' '(' ID ')'
                    | 'estado_carga' '(' ID ')'
                    | 'leer_generacion' '(' ID ')'

```

```

| 'leer_demanda' '(' ID ')'
| '(' expresion ')'
| 'no' factor

```

Los comandos de lectura son factores porque retornan valores numéricos y pueden usarse dentro de expresiones aritméticas o lógicas. Los operadores lógicos, relacionales y aritméticos se asocian según la precedencia definida.

Manejo de errores sintácticos

La función `p_error` captura cualquier secuencia de tokens que no pueda derivarse de la gramática, informando el token inesperado y la línea donde ocurrió. Esto permite al operador localizar y corregir errores rápidamente.

Relación entre la gramática y la implementación

El código en Python (`lenguajeL.py`) es una materialización directa de esta especificación teórica:

- **lexer:** implementa las reglas léxicas (tokens, palabras reservadas, literales).
- **parser:** construye un AST en forma de tuplas anidadas que respetan la gramática abstracta.
- **interpretar:** recorre el AST y ejecuta las acciones sobre la simulación (red).
- **evaluar:** resuelve las expresiones recursivamente según la precedencia definida.

La separación entre léxico, sintaxis y semántica sigue el flujo clásico de un intérprete, lo que hace que el lenguaje L sea extensible (nuevos comandos solo requieren añadir tokens y reglas) y mantenible.

Programas de Ejemplo en Lenguaje L: Escenarios Operativos

Escenario A – Prevención de Fuga Térmica y Gestión de Alivio de Carga

El siguiente programa monitorea la temperatura de la batería B1 cada 2 segundos (simulado con espera implícita). Si supera los 55 °C, activa refrigeración, desconecta la carga solar y desvía el consumo industrial.

```
// 1. Inicialización del bus de datos
init_grid

// 2. Definición de parámetros operativos
temp_critica = 55.0
limite_tiempo = 5
ciclos_en_peligro = 0
monitoreo = verdadero

// 3. Bucle de vigilancia continua
mientras monitoreo == verdadero ejecutar
    // Leer el sensor físico
    temp_actual = leer_temperatura(bateria_principal)

    // 4. Evaluación de Riesgo Térmico
    si_verdadero temp_actual >= temp_critica entonces
        // --- ACTIVAR PROTOCOLO DE EMERGENCIA ---
        activar_refrigeracion(bateria_principal, verdadero)
        // Aislar ingreso de energía (paneles solares)
        conmutar_linea(linea_solar, falso)
        // Conectar red comercial de respaldo para aliviar estrés
        conmutar_linea(red_publica, verdadero)
```

```

        // Contabilizar el tiempo que la batería lleva en estado
crítico

        ciclos_en_peligro = ciclos_en_peligro + 1

        // Escalamiento: Si el peligro persiste tras 5
ciclos, disparar alarma

        si_verdadero ciclos_en_peligro >= limite_tiempo entonces

            emitir_alerta

        fin_si

    sino

        // --- CONDICIÓN SEGURA (O RECUPERADA) ---

        activar_refrigeracion(bateria_principal, falso)

        conmutar_linea(linea_solar, verdadero)

        conmutar_linea(red_publica, falso)

        // Reiniciar el contador de riesgo

        ciclos_en_peligro = 0

    fin_si

    // Pausa de 1 segundo para estabilización de los sensores

    esperar(1)

fin_mientras

```

Escenario B – Balance de Carga y Optimización Energética Autónoma

Este script evalúa el estado de carga de la batería B_principal. Si supera el 90% y la generación solar excede la demanda, inyecta excedente a la red pública; si baja del 20% en horas nocturnas, aísla sectores no esenciales.

```

init_grid

// Definición de umbrales operativos

limite_optimo = 90.0

limite_critico = 20.0

// Bucle principal de monitoreo continuo
mientras verdadero == verdadero ejecutar

    // Lectura de sensores mediante variables locales

    carga_actual = estado_carga(banco_litio)

    generacion_solar = leer_generacion(linea_solar)

    demanda_interna = leer_demanda(sector_industrial)

    // ===== CASO 1: EXCEDENTE DE ENERGÍA Y VENTA =====

    si_verdadero carga_actual >= limite_optimo y generacion_solar >
demanda_interna entonces

        conmutar_linea(red_publica, verdadero)

    fin_si

    // ===== CASO 2: DÉFICIT CRÍTICO Y AISLAMIENTO =====

    si_verdadero carga_actual <= limite_critico entonces

        // Desconectar venta para priorizar consumo interno

        conmutar_linea(red_publica, falso)

        // Racionamiento: Aislar cargas no críticas

        conmutar_linea(sector_no_esencial, falso)

        // Asegurar soporte vital de la planta

        conmutar_linea(sector_esencial, verdadero)

        // Notificación HMI y sirenas

        emitir_alerta

    fin_si

    // Pausa de estabilización del bus de datos (5 segundos)

```

```
    esperar(5)  
fin_mientras
```


Conclusión

El desarrollo de software moderno ha superado el debate sobre cuál paradigma es el "correcto", centrándose en cambio en la madurez técnica para combinar las mejores herramientas de cada enfoque. La industria actual exige sistemas escalables y mantenibles, lo que ha impulsado la convergencia de paradigmas funcionales, orientados a objetos y concurrentes en lenguajes de amplio propósito como Python, JavaScript y Rust. El análisis comparativo práctico demuestra que, si bien la gestión de memoria y las reglas morfológicas varían profundamente entre los distintos lenguajes compilados e interpretados, la lógica algorítmica subyacente puede estandarizarse eficazmente. Por otro lado, la conceptualización y diseño del "Lenguaje L" evidencia que los lenguajes de propósito general no siempre son la solución ideal para todos los entornos. La construcción de un Lenguaje de Dominio Específico (DSL) como un producto arquitectónico permite encapsular la complejidad del hardware, ofreciendo a los operadores humanos una interfaz segura, acotada y adaptada estrictamente a las necesidades operativas de un sistema ciberfísico.

Referencias

Learn ⚡ ZIG Programming Language. (s. f.). Recuperado de <https://ziglang.org/learn/>

Aprende rust. (s. f.). Recuperado de <https://rust-lang.org/es/learn/>

Welcome to Python.org. (s. f.). Recuperado de <https://www.python.org/doc/>

JavaScript | MDN. (2026, 29 mayo). Recuperado de

<https://developer.mozilla.org/es/docs/Web/JavaScript>

Gotechflash, & Gotechflash. (2025, 20 noviembre). Top 5 emerging programming paradigms

Every developer should know. Recuperado de <https://gotechflash->

[com.translate.goog/top-emerging-programming-paradigms-every-developer-should-](https://gotechflash-com.translate.goog/top-emerging-programming-paradigms-every-developer-should-know/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)

[know/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc](https://gotechflash-com.translate.goog/top-emerging-programming-paradigms-every-developer-should-know/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)

Miriam. (2026, 24 febrero). ¿Qué son los paradigmas de programación? Recuperado de

<https://profile.es/blog/que-son-los-paradigmas-de-programacion/>

Domain-specific languages – Follow @mikhail_barash. (s. f.). Recuperado de <https://dsl-->

[course-](https://dsl--course-)

[org.translate.goog/?_x_tr_sch=http&_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_p](https://dsl--course-org.translate.goog/?_x_tr_sch=http&_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_p)

[to=tc](https://dsl--course-org.translate.goog/?_x_tr_sch=http&_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_p)